
Simulatore Sistema a Code C3

PROGETTO RETI E TELECOMUNICAZIONI

1 giugno 2026

Davide Bonsembiante - mat. 1086863
Alessandro Biscaro - mat. 1087892
Alessandro Rocco - mat. 1081179

Indice

1	Introduzione	2
2	Architettura e Scelte Implementative	3
2.1	Evoluzione rispetto al Modello Base $M/M/1$	3
2.2	Gestione degli Eventi — <code>Event.hpp</code>	3
2.3	Modello del Servente — <code>Server.hpp</code>	3
2.4	Motore di Simulazione — <code>Simulator.hpp</code> / <code>.cpp</code>	4
2.5	Calcolo delle Metriche	4
3	Logiche di Instradamento	5
4	Testing e Campagna Sperimentale	6
5	Analisi dei Risultati	7
5.1	Tempo Medio di Permanenza (W)	7
5.2	Indice di Sbilanciamento (U)	8
5.3	Parametrizzazione della Simulazione: Ambienti Eterogenei	8
6	Conclusioni	9

1. Introduzione

Il progetto **C3 Queueing System Simulator** è un'applicazione orientata allo studio prestazionale di un modello teorico di code parallele. Il sistema è costituito da N server indipendenti, ciascuno dotato di una propria coda FIFO (*First-In-First-Out*) di capacità teoricamente infinita.

Assunzioni del Modello

Il modello stocastico adottato soddisfa le seguenti ipotesi:

- Gli arrivi al sistema sono regolati da un processo di Poisson con tasso globale λ .
- Ogni pacchetto, appena giunto nel sistema, viene instradato verso una delle N code senza possibilità di preemption né di cambio coda successivo (*no jockeying*).
- Il tempo di servizio per il server i -esimo segue una distribuzione esponenziale negativa di media $1/\mu_i$.

L'obiettivo è valutare e confrontare tre distinte politiche di instradamento in termini di tempo medio di permanenza e indice di bilanciamento del carico, con un'implementazione robusta e ben documentata.

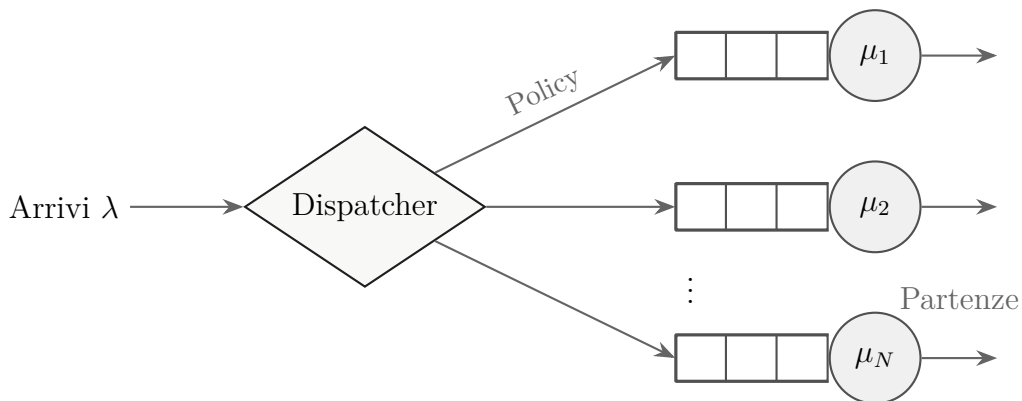


Figura 1. Modello architetturale del sistema a code parallele $M/M/N$ implementato.

2. Architettura e Scelte Implementative

L'applicazione è stata sviluppata interamente in **C++** con paradigma Object-Oriented (OO) ed è contenuta nella cartella sorgente `code/`. È stato adottato un approccio noto come *Discrete Event Simulation* (DES), in cui lo stato del sistema evolve unicamente in corrispondenza di eventi discreti schedulati.

2.1. Evoluzione rispetto al Modello Base $M/M/1$

Rispetto al codice procedurale di esempio per una singola coda $M/M/1$ analizzato a lezione, questa architettura introduce diverse estensioni fondamentali:

- **Generalizzazione Strutturale:** Il sistema non modella più un singolo server, ma ospita un vettore dinamico di N oggetti **Server**, ciascuno dotato del proprio generatore di tempi di servizio esponenziali per la stima indipendente delle partenze.
- **Componente di Routing (Dispatcher):** L'evento **ARRIVAL** non viene più semplicemente accodato allo stesso nodo. Prima deve attraversare una logica di decisione (*policy*) che interroga lo stato locale (Round-Robin) o globale (Shortest Queue) del sistema per operare lo smistamento ottimale a runtime.
- **Metriche Distribuite:** L'integrazione a scala temporale (*step-function*) per la lunghezza media della coda, che prima avveniva per un'unica entità, ora viene calcolata perentoriamente in via indipendente su ogni singola istanza **Server**. Ciò ha reso possibile l'estrazione di indici relazionali (es. sbilanciamento U) al termine della simulazione.

2.2. Gestione degli Eventi — **Event.hpp**

Ogni cambiamento di stato è modellato come un evento, incapsulato nella **struct Event**. Esistono due tipi di eventi, denotati dall'enum **EventType**:

- **ARRIVAL** — arrivo di un nuovo pacchetto dall'esterno.
- **DEPARTURE** — terminazione del servizio da parte di un dato server.

Affinché gli eventi siano estratti in rigoroso ordine cronologico, **Event** espone l'overload dell'operatore ' $<$ ' invertito. Inserendo gli eventi in una `std::priority_queue`, si ottiene un *min-heap* basato sul timestamp.

2.3. Modello del Server — **Server.hpp**

La struttura **Server** memorizza lo stato del singolo nodo, la sua coda specifica (una `std::queue<double>` contenente i tempi di arrivo dei pacchetti in attesa) e le

seguenti metriche di accumulazione:

- `total_packets_served` — contatore cumulativo dei pacchetti completati.
- `total_response_time` — somma dei tempi di permanenza su quel nodo.
- `area_queue_length` — integrale temporale della dimensione della coda, utilizzato per derivare la lunghezza media tramite il Teorema del Valore Medio.

```

1 // Aggiorna l'integrale della coda calcolando l'area del
   gradino
2 void update_area(double current_time) {
3     area_queue_length += queue.size() * (current_time -
   last_update_time);
4     last_update_time = current_time;
5 }

```

Listing 1. Metodo di calcolo dell'integrale della coda nel tempo (Server.hpp).

2.4. Motore di Simulazione — Simulator.hpp / .cpp

La classe `Simulator` è il nucleo operativo del progetto e gestisce tre responsabilità principali:

1. **Generazione di Variabili Casuali:** Sfrutta l'header `<random>` del C++. Un generatore Mersenne Twister (`std::mt19937`) alimenta distribuzioni esponenziali distinte: una per gli arrivi (tasso λ) e N per i servizi (ciascuna con il proprio μ_i).
2. **Ciclo Principale (`run()`):** Schedula il primo arrivo, quindi entra in un ciclo che termina al superamento del parametro `max_time`. A ogni iterazione, estrae l'evento imminente e invoca `handle_arrival` o `handle_departure`, avanzando il *clock* della simulazione e aggiornando le code.
3. **Robustezza sull'Input:** Se vengono forniti meno valori di μ_i rispetto a N , le entry mancanti vengono inizializzate automaticamente al valore di *fall-back* $\mu_i = 1.0$ (come documentato nei commenti del sorgente).

2.5. Calcolo delle Metriche

I KPI richiesti sono derivati rigorosamente durante il ciclo di simulazione:

- **Tempo di permanenza (W):** All'atto di una partenza, si recupera l'istante di arrivo del pacchetto al nodo. La differenza rispetto al clock corrente costituisce la latenza effettiva; la media si calcola dividendo la somma cumulata per il numero totale di partenze.

- **Lunghezza media della coda:** A ogni modifica dello stato di un `Server`, l'intervallo trascorso Δt viene moltiplicato per la dimensione istantanea precedente della coda e sommato all'integrale. Dividendo infine l'area globale per il tempo di simulazione si ottiene la lunghezza media esatta tramite il metodo *step-function*.
- **Indice di sbilanciamento:** Differenza formale tra la lunghezza media massima e minima tra gli N server al termine della run:

$$U = \max_i \bar{L}_i - \min_i \bar{L}_i$$

3. Logiche di Instradamento

Il metodo `Simulator::route_packet()` implementa le tre policy selezionabili da linea di comando:

- **Random:** Estrae un intero da una `std::uniform_int_distribution<int>(0, N-1)`. Costo algoritmico: $O(1)$.
- **Round-Robin:** Mantiene un indice ciclico persistente nella classe, incrementato tramite aritmetica modulare `(round_robin_index + 1) % N`. Costo algoritmico: $O(1)$.
- **Shortest Queue (JSQ):** Applica una scansione *greedy* lineare tra i server a ogni arrivo, restituendo l'indice del server con il minor numero di richieste accodate, tenendo conto anche del pacchetto attualmente in elaborazione. Costo algoritmico per pacchetto: $O(N)$.

Al fine di garantire trasparenza implementativa, di seguito è riportato il codice C++ del metodo `route_packet()` che illustra l'instradamento vero e proprio e la risoluzione degli stati di occupazione per il JSQ:

```

1  int Simulator::route_packet() {
2      switch (policy) {
3          case RoutingPolicy::RANDOM:
4              return random_server_dist(rng);
5
6          case RoutingPolicy::ROUND_ROBIN: {
7              int selected = round_robin_index;
8              round_robin_index = (round_robin_index + 1) %
N;
9              return selected;
10         }
11     }

```

```

12     case RoutingPolicy::SHORTEST_QUEUE: {
13         int shortest = 0;
14         // Valuta sia la coda che l'eventuale
15         pacchetto in servizio
16         size_t min_len = servers[0].queue.size() + (
17         servers[0].is_busy ? 1 : 0);
18         for (int i = 1; i < N; ++i) {
19             size_t current_len = servers[i].queue.size
20             () + (servers[i].is_busy ? 1 : 0);
21             if (current_len < min_len) {
22                 min_len = current_len;
23                 shortest = i;
24             }
25         }
26         return shortest;
27     }
28 }

```

Listing 2. Logica di selezione del servente per un nuovo arrivo (Simulator.cpp).

4. Testing e Campagna Sperimentale

Per valutare e confrontare le prestazioni delle tre politiche di instradamento al variare del carico, è stata condotta una campagna di test sistematica ed estesa basata sull'esecuzione del simulatore C++. La procedura di raccolta e analisi dei dati segue questi passi:

1. **Compilazione dell'eseguibile:** Viene compilato l'eseguibile `simulator` ottimizzato mediante il comando `make`.
2. **Esecuzione parametrica con Intervalli di Confidenza:** Si modula il tasso di arrivo $\lambda \in [5, 14]$ su un sistema base simmetrico composto da $N = 3$ serventi aventi ciascuno $\mu_i = 5$, testando ciascuna delle tre politiche. Al fine di mitigare la varianza stocastica insita nei processi markoviani e garantire validità statistica, vengono eseguite $K = 15$ simulazioni indipendenti per ogni singola configurazione. Da ciascun gruppo di run viene stimata la media campionaria e calcolato l'**Intervallo di Confidenza al 95%** sfruttando l'errore standard.

3. **Elaborazione dei Risultati e Grafici:** I valori medi di tempo di permanenza (W) e di sbilanciamento (U), corredati dai rispettivi intervalli di confidenza (rappresentati graficamente tramite barre d'errore), sono stati organizzati e tracciati per un'analisi comparativa immediata.

5. Analisi dei Risultati

Le figure seguenti illustrano i dati rilevati dalla campagna sperimentale per $\lambda \in [5, 14]$, con capacità totale del sistema $\sum_i \mu_i = 15$.

5.1. Tempo Medio di Permanenza (W)

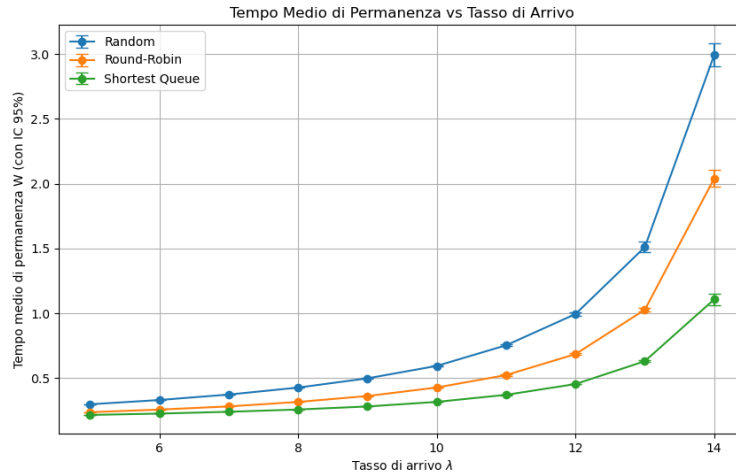


Figura 2. Tempo medio di permanenza nel sistema in funzione del tasso di arrivo λ .

Il grafico in Figura 2 rivela l'impatto critico dell'algoritmo di assegnazione al saturarsi del sistema ($\sum \mu_i = 15$). In regimi a traffico basso o moderato, *Round-Robin* si avvicina sensibilmente alle prestazioni di JSQ. Tuttavia, ad elevati carichi ($\lambda > 10$), la politica *Random* diviene impraticabile. La **Shortest Queue** mantiene stabilmente i tempi di permanenza più contenuti: adeguandosi continuamente ai micro-sbilanciamenti in tempo reale, garantisce un utilizzo parallelo ottimale delle risorse.

5.2. Indice di Sbilanciamento (U)

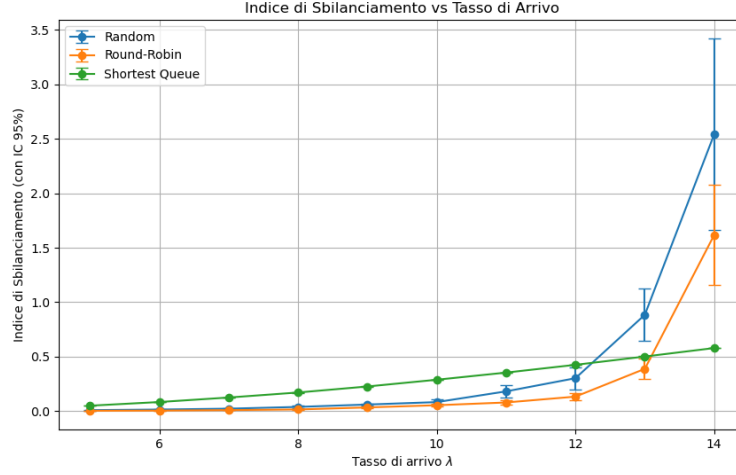


Figura 3. Indice di sbilanciamento U in funzione di λ .

La Figura 3 mostra una metrica di notevole interesse: per tassi bassi, *Round-Robin* e *Random* producono lunghezze medie statisticamente uniformi, collocandosi vicino a $U \approx 0$. In questo regime JSQ presenta uno sbilanciamento leggermente superiore, intervenendo attivamente a compensare la varianza stocastica degli arrivi.

Al contrario, avvicinandosi alla congestione ($\lambda \approx 14$), le politiche cieche collassano progressivamente. In tale regime, **JSQ** eccelle anche nello sbilanciamento, mantenendo tutte le code equamente cariche al di sotto di una soglia limitata.

5.3. Parametrizzazione della Simulazione: Ambienti Eterogenei

Per studiare il comportamento adattivo e generalizzare i concetti, è possibile alterare la simulazione parametrizzando capacità asimmetriche (eterogenee) tra i server, testando così configurazioni non banali.

Ad esempio, modificando i tassi di servizio a linea di comando da una configurazione omogenea $\mu = [5, 5, 5]$ a una sbilanciata $\mu = [7, 5, 3]$ (mantenendo identica la capacità di processamento globale $\sum \mu_i = 15$), i risultati si polarizzano. In tale scenario, policy cieche come *Round-Robin* o *Random* falliscono molto prima, poiché smistano un terzo del traffico incondizionatamente alla velocità del nodo. Il server "lento" ($\mu_3 = 3$) entra rapidamente in overflow e diventa un insormontabile collo di bottiglia.

Viceversa, la **Shortest Queue** rivela in questa configurazione il suo reale vantaggio: il server rapido ($\mu_1 = 7$) smaltisce pacchetti velocemente, rimanendo con la coda frequentemente vuota. Il *Dispatcher JSQ*, osservando l'assenza di congestione, indirizzerà in automatico e naturalmente un volume maggiore di traffico verso questo nodo di fascia alta. Il sistema implementa così una vera e propria forma di

Load-Balancing Dinamico, autoadattandosi alla potenza di calcolo disomogenea e garantendo stabilità QoS per l'intera sovrastruttura.

6. Conclusioni

L'implementazione in C++ si è dimostrata accurata sotto il profilo stocastico e computazionalmente efficiente: le simulazioni ad alta densità di eventi vengono completate in decimi di secondo. Le assunzioni teoriche sui modelli $M/M/1$ accoppiati risultano pienamente supportate:

- La politica **Shortest Queue (JSQ)**, reattiva e deterministica, migliora radicalmente i tempi di risposta (QoS) e ritarda la saturazione in condizioni di traffico elevato.
- La politica **Round-Robin** rappresenta l'alternativa *lightweight* preferibile quando le risorse algoritmiche o la latenza di ispezione risultino vincolate.